

TOX Flyer coding guidelines

Asis Hallab, “el Bobito”

May 6, 2025

1 Rendering Abstraction in the TOX Flyer

This section introduces a light-weight rendering abstraction strategy for the TOX Flyer, based on the current Babylon.js implementation. The aim is not to introduce unnecessary complexity, but to gently decouple rendering logic from application logic in a way that respects the KISS principle and anticipates future portability needs (e.g., migration to WebGPU).

1.1 Current Implementation Pattern

In the present implementation, Babylon-specific drawing commands are used directly inside the main application logic. For instance:

```
const sphere = BABYLON.MeshBuilder.CreateSphere("s", {diameter: 1}, scene);
sphere.position = new BABYLON.Vector3(x, y, z);
sphere.material = new BABYLON.StandardMaterial("mat", scene);
sphere.material.diffuseColor = new BABYLON.Color3(1, 0, 0);
```

This pattern works well for fast development but makes it difficult to:

- Replace Babylon.js with another engine in the future (e.g., raw WebGPU).
- Test scene logic independently of Babylon’s rendering behavior.
- Provide reusable components across the Flyer or future viewers.

1.2 Proposed Abstraction Pattern

The following pattern introduces a thin, declarative rendering interface that encapsulates Babylon-specific logic while remaining simple and readable.

Example: Minimal Rendering Interface

`render_interface.js`:

```
export function addSphere(scene, position, color, label) {
  const sphere = BABYLON.MeshBuilder.CreateSphere(label, {diameter: 1}, scene);
  sphere.position = new BABYLON.Vector3(...position);
  sphere.material = new BABYLON.StandardMaterial("mat", scene);
  sphere.material.diffuseColor = BABYLON.Color3.FromHexString(color);
  return sphere;
}
```

Usage in Application Code

```
import { addSphere } from "./render_interface.js";

const s = addSphere(scene, [x, y, z], "#ff0000", "module_17");
```

Benefits

- Babylon logic is isolated in one file.
- Scene logic becomes declarative and easier to read.
- Future refactoring (e.g., to WebGPU or three.js) only requires changes in the interface layer.

1.3 Scope and Philosophy

This rendering abstraction is:

- **Not a framework:** No additional logic or state management is introduced.
- **Not a rewrite:** Existing drawing commands are simply wrapped.
- **Not a barrier to speed:** Developers may still use Babylon directly when rapid testing is needed.

It is a lightweight buffer between creative experimentation and technical future-proofing — aligned with F42 principles and designed to scale with the increasing complexity of the TOX Flyer.

1.4 Conclusion

Adopting this minimal RAL allows the TOX Flyer to remain agile while building a foundation for long-term reuse and interoperability. It ensures the Flyer can later be re-targeted (e.g., to WebGPU) without re-engineering all application logic, while preserving the clarity and joy of its current Babylon-based design.

2 Unified Parallel Loop Snippet

To streamline parallel execution across different computing environments, we define a unified Fortran snippet that automatically generates precompiler code enabling compilation to one of three execution modes: **Coarrays**, **OpenMP**, or **serial fallback**. This is achieved using preprocessor directives that adapt the loop structure based on compiler flags. The loop body itself remains unchanged, ensuring that the parallelization mechanism is orthogonal to the logic being executed.

Below is the result of expanding the `f42:do-parallel` snippet, which selects the appropriate backend at compile time:

```
! -----
! START OF PARALLEL LOOP
! -----
# ifdef COARRAY
start_i = (this_image() - 1) * N / num_images() + 1
end_i   = min(this_image() * N / num_images(), N)

do i = start_i, end_i
# endif

# ifdef OPENMP
!$omp parallel do private(i)
do i = 1, N
# endif

#if !defined(COARRAY) && !defined(OPENMP)
do i = 1, N
#endif
```

```

! BODY OF THE PARALLEL LOOP
! -----

! -----
! END BODY OF THE PARALLEL LOOP

end do ! END OF DO-LOOP

# ifdef COARRAY
sync all
# endif

# ifdef OPENMP
!$OMP END PARALLEL
# endif

! -----
! END OF PARALLEL LOOP
! -----

```

Compile with:

- -DUSE_OPENMP and -fopenmp for OpenMP
- -DUSE_COARRAY for Coarrays (e.g. with -fcoarray=single for shared memory, or -fcoarray=lib for distributed)